Department of

Artificial Intelligence & Machine Learning

B.Tech III Semester

Academic Year 2026-2027

Data Base Management Systems (25B11CS001)

Capstone Stone Project Documentation



ADITYA UNIVERSITY

Surampalem, Kakinada District

Andhra Pradesh-533437

SOFTWARE MANAGEMENT SYSTEM

A DBMS Capstone Project Report submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted By

C. JAGA JEEVAN - 25B11AI472

P.BALARKANANDA - 25B11AI876

R.TEJA - 25B11AIA23

B.VENKATESH - 25B11AI136

Under the esteemed guidance of

G L NARASAMBA VANGURI, M. Tech, (Ph. D).

Assistant Professor

Department of AI&ML



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

ADITYA UNIVERSITY

(Formerly Aditya Engineering College (A))

Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh, India

2026 - 2027

ADITYA UNIVERSITY

SCHOOL OF COMPUTING

Department of AI&ML

CERTIFICATE

This is to certify that the Cornerstone project work entitled *"SOFTWARE MANAGEMENT SYSTEM"* is submitted by

CH. JAGA JEEVAN	**(25B11AI472)**
P. BALARKANANDA	**(26B21AI876)**
R. TEJA	**(26B21AIA23)**
B. VENKATESH	**(26B21AI136)**

in partial fulfillment of the requirements for the award of the B.Tech. degree in AI&ML for the academic year 2026-2027.

Project Guide
Mrs. G L Narasamba vanguri , M.Tech.,(Ph.D),
Assistant Professor,
Department of AI&ML

Head of the Department
Dr.Naga bhargavi, Ph.D,
Associate Professor,
Department of AI&ML

DECLARATION

We hereby declare that the DBMS Capstone Project entitled "Software Management System" is a genuine work carried out by us. The database design, SQL implementation, queries and documentation presented in this report are our own work, completed under the guidance of Ms G L NARASAMBA VANGURI, Department of Artificial Intelligence and Machine Learning, Aditya University, Surampalem. We further declare that this work has not been submitted, in full or in part, for the award of any degree of this or any other educational institution.

Project Associates

CH. JAGA JEEVAN **(25B11AI472)**

P. BALARKANANDA **(26B21AI876)**

R. TEJA **(26B21AIA23)**

B. VENKATESH **(26B21AI136)**

Date: 14 September 2026

Place: Surampalem

ACKNOWLEDGEMENT

It is with immense pleasure that we express our indebted gratitude to our project guide Mr. T. Srinivasulu, Department of Artificial Intelligence and Machine Learning, who guided us throughout the capstone project. We are thankful for the constant encouragement and valuable suggestions received at every review stage.

We are grateful to Dr. K. Naga Bhargavi, Head of the Department, for inspiring us and for arranging the facilities and resources needed for this project.

We extend our sincere thanks to the Management, Dr. M. Sreenivasa Reddy, Deputy Pro-Chancellor, Dr. M. B. Srinivas, Vice-Chancellor, and Dr. A. Ramesh, Pro Vice-Chancellor (Engineering & Sciences), for providing an excellent academic environment.

Not to forget, we thank the Teaching and Non-Teaching Staff, our parents and our friends, who have directly or indirectly supported us in completing this project on time.

Project Associates

CH. JAGA JEEVAN (25B11AI472)

P. BALARKANANDA (26B21AI876)

R. TEJA (26B21AIA23)

B. VENKATESH (26B21AI136)

ADITYA UNIVERSITY

Surampalem

Department of Computer Science and Engineering

DBMS CAPSTONE PROJECT

Project Documentation Report

Project Title: Software Management System

Submitted by:

S.No	Student Name	Roll Number
1	P BALARKANANDA	25B11AI876
2	R.TEJA	25B11AIA23
3	B.VENKATESH	25B11AI136
4	C JAGA JEEVAN	25B11AI472

Course: Database Management System (DBMS) Course Code: DBMS

Faculty Coordinator:

Academic Year: 2026–27 Date of Submission: 14 September 2026

GitHub Repository Link: https://github.com/pbalarkananda-cpu/SOFTWARE-MANAGEMENT-SYSYTEM.git

1. ABSTRACT

The Software Management System is a relational database project designed to manage software projects, clients, employees, modules, tasks, and issue tracking in one organized system.

The system consists of six main entities: Client, Employee, Project, Module, Task, and Issue, connected through foreign-key relationships to maintain data consistency across project planning, assignment, monitoring, and support activities.
integrity.

Built using **SQL** and normalized up to **Third Normal Form (3NF)**, the system uses **NOT NULL, UNIQUE, and CHECK constraints** to maintain accurate and valid data.

The system supports customer management, savings/current/fixed-deposit accounts, deposits, withdrawals, transaction recording, and loan tracking. SQL queries demonstrate **joins, aggregate functions, GROUP BY/HAVING, subqueries, and reporting operation**

TABLE OF CONTENTS

Chapter	Title	Page No.
	Certificate	i
	Declaration	ii
	Acknowledgement	iii
	Project Documentation Report	iv
1	**ABSTRACT**	v
	List of Figures	vii
	List of Tables	vii
2	**INTRODUCTION**	1
	2.1 Background	1
	2.2 Purpose of the Database	1
3	**PROBLEM STATEMENT**	1
4	**OBJECTIVES**	1
5	**TOOLS & TECHNOLOGIES USED**	2
6	**DATABASE DESIGN**	2
	6.1 Identified Entities	2
	6.2 Entity Attributes	2
	6.3 ER Diagram	4
	6.4 Relationships & Cardinality	4
	6.5 Relational Database Schema	4
	6.6 Normalization	6
7	**DATABASE IMPLEMENTATION**	8
	7.1 DDL — Database and Table Creation	8
	7.2 Sample Data — DML.	10
	7.3 Constraints & Data Integrity	10
	7.4 SQL Operations & Queries	10
	7.5 Sample Outputs / Screenshots	12
8	**MODULES & FUNCTIONALITIES**	14
9	**GITHUB REPOSITORY & TEAM RESPONSIBILITIES**	14
10	**ENHANCEMENTS & FUTURE IMPROVEMENTS**	15
11	**CONCLUSION**	15
12	**REFERENCES**	15
	APPENDIX	16
	A. Complete SQL Script	16
	B. Additional Screenshots / Outputs	16

LIST OF FIGURES

Figure No.	Title of the Figure	Page No.
6.3	ER Diagram	4
Figure 1	Employee records / basic SELECT output	12
Figure 2	In-progress projects / filtering output	12
Figure 3	Aggregate or GROUP BY output	13
Figure 4	Join or subquery output	13

Table No.	**Title of the Table**	**Page No.**
5	TOOLS & TECHNOLOGIES USED	2
6.1	Identified Entities	2
6.2	Entity Attributes	2
6.4	Relationships & Cardinality	4
6.5	Relational Database Schema	4
7.2	Sample Data — DML.	10
7.3	Constraints & Data Integrity	10
7.4	SQL Operations & Queries	10
8	MODULES & FUNCTIONALITIES	14
9	GITHUB REPOSITORY & TEAM RESPONSIBILITIES	14

2. INTRODUCTION

2.1 Background

Software organizations handle structured information related to clients, employees, projects, modules, development tasks, deadlines, issue reports, and delivery status. A relational database is required to store this information consistently and to maintain relationships between the different records. In this project, the software management domain is represented using six related tables: Client, Employee, Project, Module, Task, and Issue. The schema separates data into appropriate entities and uses keys and constraints to reduce duplication and protect data integrity. SQL queries are then used to retrieve, update, aggregate, and analyse the stored information.

2.2 Purpose of the Database

The database provides a centralized structure for managing client details, employee profiles, project schedules, module ownership, assigned tasks, and reported software issues. It can support project managers, team leads, developers, testers, and administrators who need to retrieve project/task information, track employee workload, monitor delivery progress, record issue resolution, and generate summary information. The relationships between the tables allow related information to be accessed through SQL joins and subqueries instead of maintaining disconnected records.

3. PROBLEM STATEMENT

Manual or spreadsheet-based project tracking can lead to duplicate client records, unclear task ownership, inconsistent project status, and difficulty in monitoring software defects.

The proposed system centralizes client, employee, project, module, task, and issue information into a single relational database.

The system provides structured data management, referential integrity, validation and SQL-based reporting for efficient software project planning and monitoring.

4. OBJECTIVES

- Objective 1: Design a relational schema containing six related software management entities.
- Objective 2: Implement primary keys, foreign keys, NOT NULL, UNIQUE, CHECK, and DEFAULT constraints for data integrity.
- Objective 3: Populate the database with representative client, employee, project, module, task, and issue records.
- Objective 4: Demonstrate SQL retrieval, updates, joins, aggregate functions, GROUP BY/HAVING, and subqueries.
- Objective 5: Document the database design, implementation, sample data, queries, and team responsibilities.

5. TOOLS & TECHNOLOGIES USED

Category	Tool / Technology Used
Database Server	SQL-compatible relational database
Query / Admin Tool	SQL client / SQL environment
Front-end (if any)	Not specified in the uploaded SQL files
Operating System	Not specified in the uploaded files
Version Control	GitHub

6. DATABASE DESIGN

6.1 Identified Entities

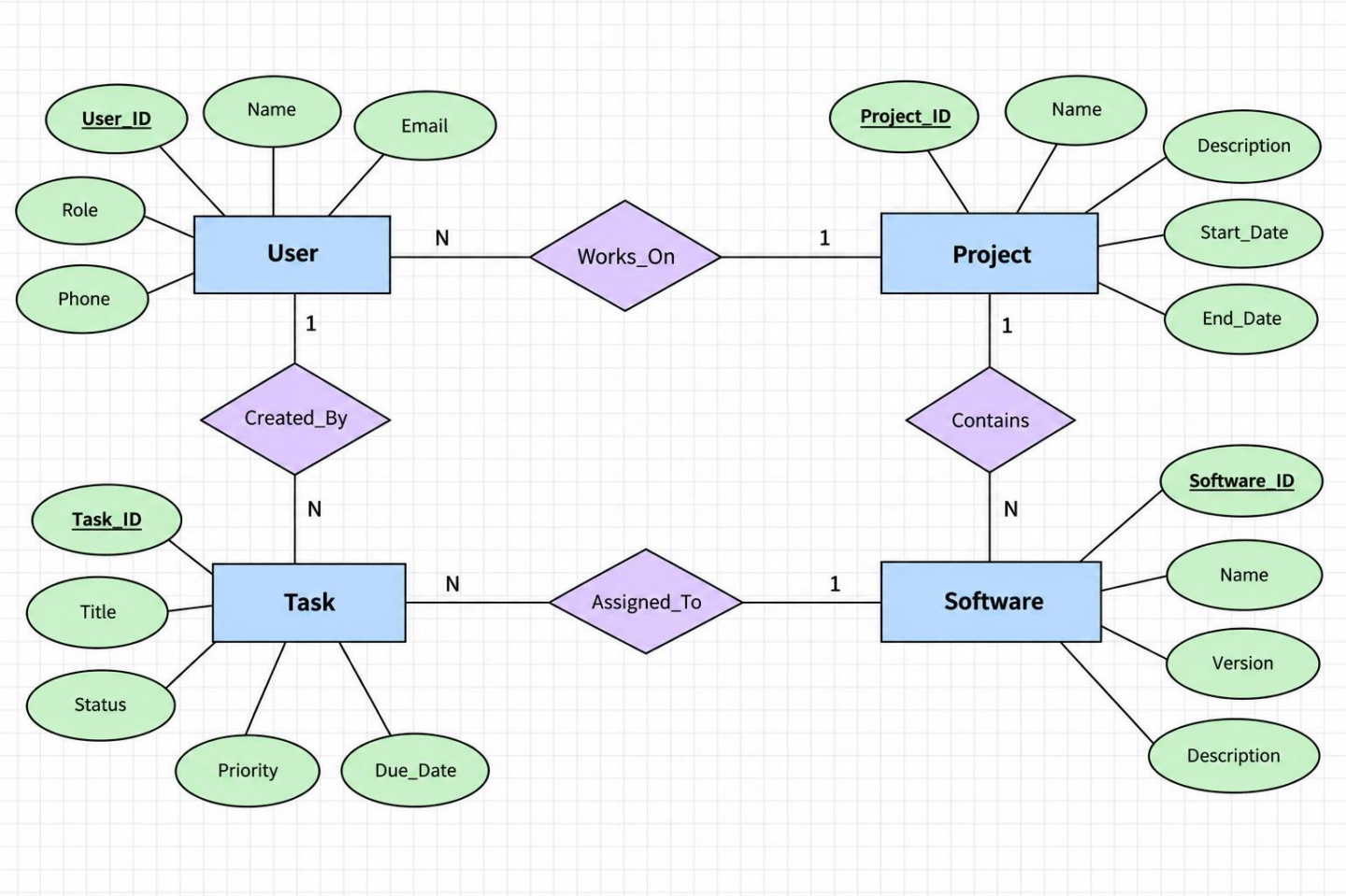
S.No	Entity Name	Description
1	Client	Stores client organization details, contact person, email, phone, city, and creation date.
2	Employee	Stores employee identity, role, department, contact details, hire date, and working status.
3	Project	Stores software project details, client ownership, manager assignment, schedule, budget, and status.
4	Module	Stores project modules, lead employee, priority, due date, and development status.
5	Task	Stores task assignments, task titles, estimated/actual hours, due dates, and progress status.
6	Issue	Stores software issue reports, severity, status, reporting employee, dates, and resolution notes.

6.2 Entity Attributes

Entity	Attribute	Data Type	Key (PK/FK/-)	Attribute Type
Client	ClientID	INTEGER	PK	Simple
Client	ClientName	VARCHAR(80)	-	Simple
Client	ContactPerson	VARCHAR(60)	-	Simple
Client	Email	VARCHAR(100)	-	Simple
Client	Phone	VARCHAR(15)	-	Simple
Client	City	VARCHAR(50)	-	Simple
Client	CreatedAt	DATE	-	Derived/Default
Employee	EmployeeID	INTEGER	PK	Simple
Employee	FirstName	VARCHAR(30)	-	Simple
Employee	LastName	VARCHAR(30)	-	Simple
Employee	Role	VARCHAR(40)	-	Simple
Employee	Department	VARCHAR(50)	-	Simple
Employee	Email	VARCHAR(100	-	Simple

		)		
Employee	Phone	VARCHAR(15)	-	Simple
Entity	Attribute	Data Type	Key (PK/FK/-)	Attribute Type
Employee	HireDate	DATE	-	Simple
Employee	Status	VARCHAR(15)	-	Derived/Default
Project	ProjectID	INTEGER	PK	Simple
Project	ClientID	INTEGER	FK	Simple
Project	ProjectName	VARCHAR(100)	-	Simple
Project	StartDate	DATE	-	Simple
Project	EndDate	DATE	-	Simple
Project	Budget	DECIMAL(12,2)	-	Simple
Project	Status	VARCHAR(20)	-	Derived/Default
Project	ManagerID	INTEGER	FK	Simple
Module	ModuleID	INTEGER	PK	Simple
Module	ProjectID	INTEGER	FK	Simple
Module	ModuleName	VARCHAR(80)	-	Simple
Module	LeadEmployeeID	INTEGER	FK	Simple
Module	Priority	VARCHAR(15)	-	Simple
Module	Status	VARCHAR(20)	-	Derived/Default
Module	DueDate	DATE	-	Simple
Task	TaskID	INTEGER	PK	Simple
Task	ModuleID	INTEGER	FK	Simple
Task	AssignedTo	INTEGER	FK	Simple
Task	TaskTitle	VARCHAR(120)	-	Simple
Task	TaskStatus	VARCHAR(20)	-	Simple
Task	EstimatedHours	DECIMAL(6,2)	-	Simple
Task	ActualHours	DECIMAL(6,2)	-	Derived/Default
Issue	IssueID	INTEGER	PK	Simple
Issue	ProjectID	INTEGER	FK	Simple
Issue	ModuleID	INTEGER	FK	Simple
Issue	ReportedBy	INTEGER	FK	Simple
Issue	IssueTitle	VARCHAR(120)	-	Simple
Issue	Severity	VARCHAR(20)	-	Simple
Issue	IssueStatus	VARCHAR(20)	-	Derived/Default
Issue	ReportedDate	DATE	-	Derived/Default
Issue	ResolvedDate	DATE	-	Simple
Issue	ResolutionNotes	VARCHAR(255)	-	Simple

6.3 ER Diagram



6.4 Relationships & Cardinality

Entity 1	Relationship	Entity 2	Cardinality (1:1 / 1:N / M:N)
Client	requests	Project	1:N
Employee	manages	Project	1:N
Project	contains	Module	1:N
Module	contains	Task	1:N
Employee	is assigned	Task	1:N
Project	has	Issue	1:N

6.5 Relational Database Schema

Table Name	Column Name	Data Type	PK/FK/-	Constraints
Client	ClientID	INTEGER	PK	PRIMARY KEY
Client	ClientName	VARCHAR(80)	-	NOT NULL
Client	ContactPerson	VARCHAR(60)	-	NOT NULL
Client	Email	VARCHAR(100)	-	NOT NULL, UNIQUE
Client	Phone	VARCHAR(15)	-	NOT NULL, CHECK length >= 10
Client	City	VARCHAR(50)	-	NULL allowed
Client	CreatedAt	DATE	-	DEFAULT CURRENT_DATE
Employee	EmployeeID	INTEGER	PK	PRIMARY KEY
Employee	FirstName	VARCHAR(30)	-	NOT NULL
Employee	LastName	VARCHAR(30)	-	NOT NULL
Employee	Role	VARCHAR(40)	-	NOT NULL
Employee	Department	VARCHAR(50)	-	NOT NULL
Employee	Email	VARCHAR(100)	-	NOT NULL, UNIQUE
Employee	Phone	VARCHAR(15)	-	NOT NULL
Employee	HireDate	DATE	-	NOT NULL
Employee	Status	VARCHAR(15)	-	DEFAULT Active, CHECK allowed statuses
Project	ProjectID	INTEGER	PK	PRIMARY KEY
Project	ClientID	INTEGER	FK	NOT NULL, FOREIGN KEY -> Client
Project	ProjectName	VARCHAR(100)	-	NOT NULL
Project	StartDate	DATE	-	NOT NULL
Project	EndDate	DATE	-	NULL allowed
Project	Budget	DECIMAL(12,2	-	CHECK >= 0

		)		
Project	Status	VARCHAR(20)	-	DEFAULT Planned, CHECK allowed statuses
Project	ManagerID	INTEGER	FK	NOT NULL, FOREIGN KEY -> Employee
Module	ModuleID	INTEGER	PK	PRIMARY KEY
Module	ProjectID	INTEGER	FK	NOT NULL, FOREIGN KEY -> Project
Table Name	Column Name	Data Type	PK/FK/-	Constraints
Module	ModuleName	VARCHAR(80)	-	NOT NULL
Module	LeadEmployeeID	INTEGER	FK	NOT NULL, FOREIGN KEY -> Employee
Module	Priority	VARCHAR(15)	-	CHECK Low/Medium/High/Critical
Module	Status	VARCHAR(20)	-	DEFAULT Not Started, CHECK allowed statuses
Module	DueDate	DATE	-	NULL allowed
Task	TaskID	INTEGER	PK	PRIMARY KEY
Task	ModuleID	INTEGER	FK	NOT NULL, FOREIGN KEY -> Module
Task	AssignedTo	INTEGER	FK	NOT NULL, FOREIGN KEY -> Employee
Task	TaskTitle	VARCHAR(120)	-	NOT NULL
Task	TaskStatus	VARCHAR(20)	-	CHECK Open/In Progress/Review/Done
Task	EstimatedHours	DECIMAL(6,2)	-	NOT NULL, CHECK > 0
Task	ActualHours	DECIMAL(6,2)	-	DEFAULT 0, CHECK >= 0
Task	DueDate	DATE	-	NULL allowed
Issue	IssueID	INTEGER	PK	PRIMARY KEY
Issue	ProjectID	INTEGER	FK	NOT NULL, FOREIGN KEY -> Project
Issue	ModuleID	INTEGER	FK	FOREIGN KEY -> Module
Issue	ReportedBy	INTEGER	FK	NOT NULL, FOREIGN KEY -> Employee
Issue	IssueTitle	VARCHAR(120)	-	NOT NULL
Issue	Severity	VARCHAR(20)	-	CHECK Low/Medium/High/Critical
Issue	IssueStatus	VARCHAR(20)	-	DEFAULT Open, CHECK allowed statuses

Issue	ReportedDate	DATE	-	DEFAULT CURRENT_DATE
Issue	ResolvedDate	DATE	-	NULL allowed
Issue	ResolutionNotes	VARCHAR(255)	-	NULL allowed

6.6 Normalization

UNF -> 1NF -> 2NF -> 3NF

UNF: Software management information could be represented as a large combined record containing client, employee, project, module, task, and issue details.

1NF: The design separates repeating groups and keeps attributes atomic. For example, employee names and client contact information are stored in separate fields, while tasks and issues are recorded as individual rows.

2NF: Each entity is separated into its own table with a primary key so non-key attributes depend on the complete key.

3NF: Relationships are represented using foreign keys, so client, employee, project, module, task, and issue facts are stored in their appropriate tables rather than being repeatedly duplicated. This reduces update, insertion, and deletion anomalies.

7. DATABASE IMPLEMENTATION

7.1 DDL — Database and Table Creation

```
    -- Drop tables if they exist
DROP TABLE IF EXISTS Issue; DROP TABLE
IF EXISTS Task;
DROP TABLE IF EXISTS Module;
DROP TABLE IF EXISTS Project;
DROP TABLE IF EXISTS Employee;
DROP TABLE IF EXISTS Client;


-- 1. CLIENT TABLE

CREATE TABLE Client (
    ClientID INTEGER PRIMARY KEY,
    ClientName VARCHAR(80) NOT NULL,
    ContactPerson VARCHAR(60) NOT NULL,
    Email VARCHAR(100) NOT NULL
    UNIQUE,
    Phone VARCHAR(15) NOT NULL, City
    VARCHAR(50), CreatedAt DATE
    DEFAULT CURRENT_DATE,
    CONSTRAINT chk_client_phone CHECK (LENGTH(Phone) >= 10)
);


-- 2. EMPLOYEE TABLE CREATE TABLE Employee (
    EmployeeID INTEGER PRIMARY KEY,
    FirstName VARCHAR(30) NOT NULL,
    LastName VARCHAR(30) NOT
    NULL, Role VARCHAR(40) NOT
    NULL, Department VARCHAR(50)
    NOT NULL,
    Email VARCHAR(100) NOT NULL UNIQUE, Phone VARCHAR(15) NOT NULL,
    HireDate DATE NOT NULL, Status
    VARCHAR(15) DEFAULT 'Active'
    CHECK (Status IN ('Active','Inactive','On
    Leave'))
    );



-- 3. PROJECT TABLE

CREATE TABLE Project (
    ProjectID INTEGER PRIMARY
    KEY, ClientID INTEGER NOT
    NULL, ProjectName
    VARCHAR(100) NOT NULL,
    StartDate DATE NOT NULL,
    EndDate DATE, Budget
```

```
    DECIMAL(12,2) CHECK (Budget
    >= 0),
    Status VARCHAR(20) NOT NULL
    DEFAULT 'Planned' CHECK (Status IN
    ('Planned','In Progress','On
    Hold','Completed','Cancelled')),
    ManagerID INTEGER NOT NULL,
    FOREIGN KEY (ClientID) REFERENCES
    Client(ClientID),
    FOREIGN KEY (ManagerID) REFERENCES Employee(EmployeeID)
);
```

```
--- ----------------------------------------------------------------------------------------------------------------------------------------
-- 4. MODULE TABLE
---
CREATE TABLE Module (
    ModuleID INTEGER PRIMARY KEY,
    ProjectID INTEGER NOT NULL,
    ModuleName VARCHAR(80) NOT NULL,
    LeadEmployeeID INTEGER NOT NULL,
    Priority VARCHAR(15) NOT NULL CHECK (Priority IN ('Low','Medium','High','Critical')),
    Status VARCHAR(20) NOT NULL DEFAULT 'Not Started' CHECK (Status IN ('Not
    Started','In Progress','Testing','Completed')), DueDate DATE,
    FOREIGN KEY (ProjectID) REFERENCES Project(ProjectID),
    FOREIGN KEY (LeadEmployeeID) REFERENCES
    Employee(EmployeeID)
);


-- 5. TASK TABLE

CREATE TABLE Task ( TaskID INTEGER
    PRIMARY KEY, ModuleID INTEGER NOT
    NULL, AssignedTo INTEGER NOT NULL,
    TaskTitle VARCHAR(120) NOT NULL, TaskStatus VARCHAR(20) NOT NULL CHECK
(TaskStatus IN ('Open','In Progress','Review','Done')),
    EstimatedHours DECIMAL(6,2) NOT NULL CHECK (EstimatedHours > 0),
    ActualHours DECIMAL(6,2) DEFAULT 0 CHECK (ActualHours >= 0),
    DueDate DATE,
FOREIGN KEY (ModuleID) REFERENCES Module(ModuleID), FOREIGN KEY (AssignedTo)
REFERENCES Employee(EmployeeID)
    );



-- 6. ISSUE TABLE

CREATE TABLE Issue (
    IssueID INTEGER PRIMARY
    KEY, ProjectID INTEGER NOT
    NULL, ModuleID INTEGER,
    ReportedBy INTEGER NOT
    NULL,
    IssueTitle VARCHAR(120) NOT NULL, Severity VARCHAR(20) NOT NULL CHECK (Severity
IN ('Low','Medium','High','Critical')),
    IssueStatus VARCHAR(20) NOT NULL DEFAULT 'Open' CHECK
    (IssueStatus IN ('Open','Assigned','Resolved','Closed')),
    ReportedDate DATE DEFAULT CURRENT_DATE,
    ResolvedDate DATE,
    ResolutionNotes VARCHAR(255),
    FOREIGN KEY (ProjectID) REFERENCES Project(ProjectID), FOREIGN KEY (ModuleID)
REFERENCES Module(ModuleID),
    FOREIGN KEY (ReportedBy) REFERENCES Employee(EmployeeID)
    );
```

7.2 Sample Data — DML.

Table	No. of Records	Sample Data / Screenshot Reference
Client	5	dml.sql - Client seed records
Employee	12	dml.sql - Employee seed records
Project	8	dml.sql - Project seed records
Module	18	dml.sql - Module seed records
Task	50	dml.sql - Task seed records
Issue	20	dml.sql - Issue seed records

7.3 Constraints & Data Integrity

Constraint / Validation	Applied On	Purpose	Evidence / Screenshot
PRIMARY KEY	All six tables	Uniquely identifies each record and prevents duplicate key values.	ddl.sql
FOREIGN KEY	Project, Module, Task, Issue	Maintains valid relationships between clients, employees, projects, modules, tasks, and issues.	ddl.sql
NOT NULL	Required fields across tables	Prevents missing values in required attributes.	ddl.sql
UNIQUE	Client.Email; Employee.Email	Prevents duplicate contact values.	ddl.sql
CHECK	Status, priority, severity, hours, budget, phone length	Restricts invalid or out-of-range values.	ddl.sql
DEFAULT	CreatedAt; Project.Status; Module.Status; ActualHours; IssueStatus; ReportedDate	Provides default values when not explicitly supplied.	ddl.sql

7.4 SQL Operations & Queries

.

S.No	Operation / Query	Purpose	SQL Query / Screenshot Ref.
1	Basic SELECT - all employees	Retrieve employee records.	queries.sql Q1
2	Filtered SELECT - in-progress projects	Find active projects ordered by start date.	queries.sql Q2
3	LIKE filtering	Search projects or clients by name.	queries.sql Q3
4	Task lookup	Check tasks assigned to a selected	queries.sql Q4

		employee.	
5	UPDATE task status	Move a task from Open to In Progress or Done.	queries.sql Q5
6	INSERT issue	Record a new software issue for a project/module.	queries.sql Q6
7	Issue history	Display issue history in descending reported date order.	queries.sql Q7
8	SUM aggregate	Calculate total estimated hours across all tasks.	queries.sql Q8
S.No	Operation / Query	Purpose	SQL Query / Screenshot Ref.
9	GROUP BY + aggregates	Calculate task count, average hours, and total hours by status.	queries.sql Q9
10	GROUP BY + aggregates	Calculate open issue count by severity.	queries.sql Q10
11	JOIN + GROUP BY + HAVING	Find projects whose pending task hours exceed a threshold.	queries.sql Q11
12	JOIN + GROUP BY + HAVING	Find employees assigned to more than one active task.	queries.sql Q12
13	Subquery	Find projects whose task load exceeds the average project task load.	queries.sql Q13
14	Subquery + ORDER BY + LIMIT	List issues for the project with the highest number of open tasks.	queries.sql Q14

7.5 Sample Outputs / Screenshots

Figure 1: Employee records / basic SELECT output

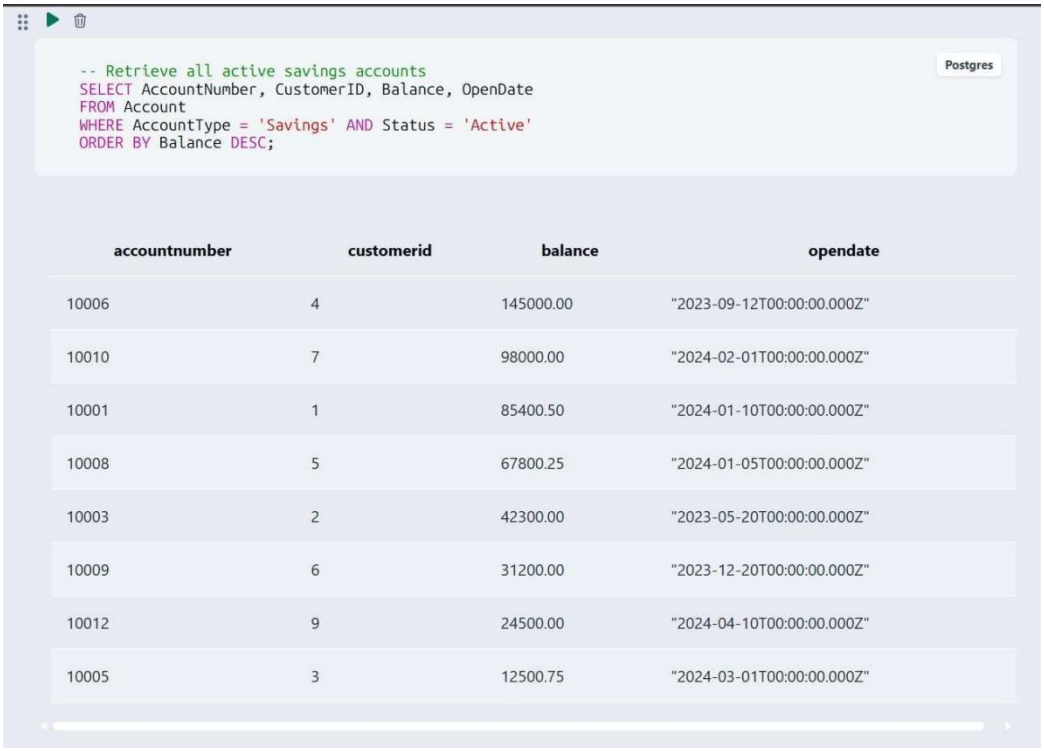


Figure 2: In-progress projects / filtering output

```sql
-- Find customers whose total account balance is higher than the average balance across all bank
SELECT CustomerID, FirstName, LastName, Email
FROM Customer
WHERE CustomerID IN (
    SELECT CustomerID
    FROM Account
    GROUP BY CustomerID
    HAVING SUM(Balance) > (SELECT AVG(Balance) FROM Account)
);
```

Postgres

customerid	firstname	lastname	email
1	Gopi	Peddinti	gopi.p@aditya.ac.in
2	Aditya	Verma	aditya.verma@example.com
4	Bhavana	Kothapalli	bhavana.k@example.com
10	Manohar	Rawat	manohar.rawat@example.com

Figure 3: Aggregate or GROUP BY output

```sql
--Average account balance grouped by Account Type
SELECT AccountType, COUNT(*) AS AccountCount, AVG(Balance) AS AverageBalance, SUM(Balance) AS TotalTypeBa
FROM Account
GROUP BY AccountType;
```

Postgres

accounttype	accountcount	averagebalance	totaltypebalance
Fixed Deposit	3	550000.000000000000	1650000.00
Savings	8	63337.687500000000	506701.50
Current	4	207500.000000000000	830000.00

Figure 4: Join or subquery output

```sql
-- Find branches where total account balance exceeds Rs. 500,000
SELECT
    b.BranchName,
    SUM(a.Balance) AS TotalBalance
FROM Branch b
JOIN Account a
    ON b.BranchID = a.BranchID
GROUP BY b.BranchID, b.BranchName
HAVING SUM(a.Balance) > 500000;
```

Postgres

branchname	totalbalance
Hyderabad Corp	1563000.00
Vizag Branch	573500.00

8. MODULES & FUNCTIONALITIES

S.No	Module Name	Functionality
1	Client Management	Stores and organizes client organization details, contact person, email, phone, city, and creation information.
2	Employee Management	Maintains employee profiles, roles, departments, contact details, hire dates, and working status.
3	Project Management	Stores project ownership, manager assignment, budget, schedule, and delivery status.
4	Module Management	Tracks project modules, lead employees, priorities, due dates, and development status.
5	Task Management	Records task assignments, estimated/actual hours, due dates, and progress state.
6	Issue Tracking & Analysis	Stores issue reports and supports defect analysis through aggregate and grouped SQL queries.

9. GITHUB REPOSITORY & TEAM RESPONSIBILITIES

Repository Link: https://github.com/DBMS-2026/Software-Management-System-DBMS

Student Name	Roll Number	Role / Responsibility	Contribution Status
P.BALRAKANANDA	25B11AI876	Wrote DDL scripts for software management tables, keys, and constraints	Completed
C.JAGA JEEVAN	25B11AI472	Designed ER diagram and relational schema for client-project-task relationships	Completed
R.TEJA	25B11AIA23	Wrote CRUD, join, aggregate, and issue-tracking queries	Completed
B.VENKATESH	25B11AI136	Compiled documentation and prepared screenshot sections	Completed

10. ENHANCEMENTS & FUTURE IMPROVEMENTS

Future improvements can extend the current database with user authentication and role-based access, richer project dashboards, notification alerts for overdue tasks, sprint and milestone planning, audit logging, stored procedures/triggers where supported, indexing for frequently queried columns, and a front-end interface for project managers and team members. The uploaded SQL files do not include these additional features, so they are proposed enhancements rather than implemented features in the current submission.

11. CONCLUSION

The Software Management System successfully models the main software project entities and their relationships using a relational database structure. The DDL defines six tables with primary keys, foreign keys, NOT NULL, UNIQUE, CHECK, and DEFAULT constraints. The DML script supplies representative records for clients, employees, projects, modules, tasks, and issues. The query script demonstrates selection and filtering, status updates, task tracking, joins, aggregate functions, GROUP BY/HAVING, and subqueries. The project therefore meets its core objectives of relational design, data integrity, sample-data population, and practical SQL analysis. The team also divided the work across DDL development, ER/schema design, query development, and documentation/screenshot compilation.

12. REFERENCES

1. Uploaded project DDL script: ddl.sql.

2. Uploaded project DML/sample-data script: dml.sql.

3. Uploaded SQL query script: queries.sql.

4. Aditya University - DBMS Capstone Project documentation template.

5. GitHub repository: Software Management System (project repository).

APPENDIX

A. Complete SQL Script

The final SQL implementation is supplied through the uploaded project files:

• ddl.sql - database table definitions, keys, foreign keys, and constraints.

• dml.sql - sample data for all six tables.

• queries.sql - representative SQL operations and analytical queries.

Repository: https://github.com/gopipeddinti1986-oss/Software-Management-System.git

B. Additional Screenshots / Outputs

Additional evidence can include SQL execution screenshots showing table creation, sample data, employee/project retrieval, task status updates, aggregate results, joins, GROUP BY/HAVING results, and subquery outputs. No execution screenshots were included in the uploaded source files, so this section is intentionally left for the team to add its actual screenshots.